

Tshepiso Freddy Thosago

BCom Information Systems · NQF Level 7 · University of Johannesburg

Johannesburg, Gauteng · +27 680 409 678 · tshepisothosago@gmail.com · github.com/tshepisofrominnostation
tshepiso-thosago.netlify.app

PROFESSIONAL SUMMARY

Full-stack developer and BCom Information Systems graduate specialising in backend engineering, database architecture, and cloud deployment. Independently built and deployed **4 production web applications** across fintech, edtech, and enterprise resource planning — each solving a specific South African business problem. Proven ability to design multi-tenant PostgreSQL schemas, implement POPIA-compliant data isolation, and ship full-stack systems from requirement gathering to live deployment on Railway and Netlify. Seeking a junior-to-mid developer role where I can contribute real code from day one.

CORE TECHNICAL SKILLS

Backend	Python (Flask 3.0), Node.js, TypeScript, Express, gunicorn	Frontend	React, Next.js 14, HTML5, CSS3, JavaScript (ES6+), Tailwind
Databases	PostgreSQL, Prisma ORM, Supabase + RLS, SQLite, MySQL	APIs & Integration	REST API design, JWT auth, Meta WhatsApp API, Resend API
Cloud & DevOps	Railway, Netlify, Netlify Functions, Git, GitHub, Nixpacks	Architecture	Multi-tenant design, POPIA compliance, atomic transactions, RBAC
Analytics	Power BI, Advanced Excel, SAP Ariba, SAP Comet	Methodologies	Business Analysis, Schema Design, PRD / BRD documentation

FEATURED SOFTWARE ENGINEERING PROJECTS

Rem0Beg Pay — Earned Wage Access Platform

[Live Demo](#) [GitHub](#)

- Stack:** TypeScript · Node.js · PostgreSQL · decimal.js · Netlify · Sage / PaySpace / Pastel Integration
- Designed and deployed a B2B2C fintech platform enabling SA employees to access up to **25% of earned wages before payday** via a flat R15 fee — deliberately structured outside National Credit Act classification.
 - Engineered **atomic PostgreSQL transactions** with row-level locking to prevent double-spending when concurrent withdrawal requests arrive for the same employee.
 - Enforced **POPIA-compliant multi-tenant data isolation** — every table scoped by `company_id` extracted from JWT; zero cross-tenant data leakage by design.
 - Replaced JavaScript native floats with `decimal.js` for all financial arithmetic — preventing IEEE 754 rounding errors across thousands of transactions.
 - Built payroll integration layer supporting **Sage 300 People (OAuth2), PaySpace (API key), and Pastel (SFTP/CSV)** with quarantine logging for corrupted batch records.
 - Implemented month-end reconciliation worker: aggregates all advances per employer, deducts in bulk at payroll run, marks period as `RECONCILED`.

Dinoto Technical Secondary School — ERP System

[Live Demo](#) [GitHub](#)

- Stack:** Next.js 14 · TypeScript · Node.js · Express · Prisma ORM · PostgreSQL · JWT RBAC
- Architected a full-stack school ERP replacing paper registers — digitising textbook and asset custody across **9 modules** (Scanner, Audit, Invoices, Reports, Asset Register).
 - Designed a **14-table PostgreSQL schema** with separate `Book` and `BookCopy` entities (one title → many barcoded physical copies) and an immutable `Transaction` audit trail.
 - Implemented **role-based access control** across 6 staff levels (Super Admin → Teacher → Student) via JWT middleware; roles verified at route handler, not just UI.
 - Built barcode scan-to-issue/return workflow using **USB HID scanner input capture** — scans processed in under 200ms via Prisma `$transaction()` atomic operations.
 - Projected to save each school **R247,000+/year** in lost textbooks; national scale (23,000+ schools) represents R6.1B annual opportunity.

SIPMS — Smart Inventory & Procurement Management

[➤ Live on Railway](#)[➤ GitHub](#)

System

Stack: Python 3.11 · Flask 3.0 · PostgreSQL · SQLite · gunicorn · Railway · Jinja2

- ▶ Built and deployed a full ERP-style procurement system with **17 routes across 5 modules**: Products, Suppliers, Purchase Orders, Transactions, Reports — live on Railway with PostgreSQL backend.
- ▶ Implemented **dual-database architecture**: single codebase runs SQLite (local, zero-config) and PostgreSQL (production) via environment-based `get_db()` adapter pattern.
- ▶ Designed an **append-only transactions table** — INSERT only, never UPDATE/DELETE — providing a tamper-proof audit trail; current stock derived as SUM of all movements.
- ▶ Configured **Nixpacks build pipeline** and `railway.json` for automated Railway deployment; `DATABASE_URL` env var switches DB at runtime with no code changes.
- ▶ Gathered business requirements, authored BRDs and technical specifications; developed Power BI dashboards for procurement and supplier performance analytics.

Book-Smart — B2B SaaS School Asset Tracking Platform

[➤ Live Demo](#)[➤ GitHub](#)

Stack: JavaScript · Netlify Serverless Functions (Node.js) · Supabase (PostgreSQL) · Resend API · Meta WhatsApp API

- ▶ Built a multi-tenant serverless platform for township primary schools — **zero server maintenance, R0/month hosting** on Netlify + Supabase free tiers.
- ▶ Designed a **9-table PostgreSQL schema** with Supabase Row-Level Security — database-layer POPIA compliance; cross-school data access structurally impossible.
- ▶ Engineered barcode scan-to-custody workflow: USB HID scanners captured via browser `keydown` events, processed through Netlify serverless functions in under 200ms.
- ▶ Integrated WhatsApp Business API for automated parent billing notifications and Resend for transactional email confirmations.
- ▶ Authored full PRD, REST API specifications, and architecture documentation for the platform.

EDUCATION

University of Johannesburg (UJ)

Bachelor of Commerce (BCom) in Information Systems · NQF Level 7

Majors: Information Systems & Project Management · Minor: Entrepreneurship · Aggregate: 65%+

PERSONAL DECLARATION

Nationality: South African Citizen by birth

Portfolio: tshepiso-thosago.netlify.app

Experience: Less than 2 years formal corporate post-studies

Availability: Immediate · Open to full-time & contract